OOP FINAL EXAM REF. SHEET

vector<Base*>
↳ for(auto p:elems) ✓ copies only the address, not the object.
for(auto& q:questions) → no copy, can modify

const T& → for reading; avoids copies
T& → modify original
for(const auto& q: questions) → no copy, no modify → best for read/search
Repo& repo → original repo in service.
for(auto q:questions) → copies each element
auto v: volunteers → copy, original never changes

```
void f(Question q) COPY
void f(const Question& q)
NO COPY, NO MODIFY
void f(Question& q)
NO COPY, CAN MODIFY
```

Color of the window:
this->setStyleSheet("background-color: ...");

Transform input string (list, coma) into vector of strings:
```
std::string diseasesString = ui->lineEdit->text().toStdString();
std::vector<std::string> diseases;
std::stringstream ss(diseasesString);
std::string disease;
while(std::getline(ss, disease, ',')){
    diseases.push_back(disease);
}
```

Exact match vs Partial match (filter)
```
for(...){
if(c.getName()==name)
```
```
if(c.getName().find(nam
!= std::string::npos)
```

"Top 3" => default constructor: Volunteer()=defa
if (topVol.size()>3) topVol.resize(3);

MicrobialWorld

Work with comboBoxes: connect(ui->comboBox, &QComboBox::currentTextChanged, this, ...);
```
ui->comboBox->addItem("All");
for(... species){
  ui->comboBox->addItem(QString::fromStdString(s));
}
std::string selection = ui->comboBox->currentText().toStdString();
```

Check if substring is contained in another string:
if(dep.getDescr().find(i) != std::string::npos)
=> WE DID FIND MATCHING WORDS (interests) in descripti

Current date: #include <QDate>
```
std::string currentDate = QDate::currentDate().toString("yyyy-MM-dd").toStdString();
if(date < currentDate){
    QMessageBox::critical(this,...);
    return;
}
```

Inside load: bool status = (std::stoi(statusString) != 0);

Set window title with more fields:
this->setWindowTitle(QString::fromStdString(member.getName()+" "+
QString::fromStdString(member.getType()));

Work with spinboxes: connect(ui->spinBox, QOverload<int>::of(&QSpinBox::valueChanged), this, ...)
```
ui->spinBox->blockSignals(true);
ui->spinBox->setValue(a.getVotes());
ui->spinBox->blockSignals(false);
ui->spinBox->setEnabled(a.getUsername()!=user.getName())
```

Find a field in a line using SUBSTRINGS:
```
std::string line = selection[0]->text().toStdString();
std::string description = line.substr(0, line.find(','));
```

In constructor:
ui->spinBox->setEnabled(false); ← Q&A OOPEXAM

Work with checkBoxes: connect(ui->checkBox, &QCheckBox::toggled, ...); | &QListWidget::itemDoubleClicked,
```
std::vector<Patient> patients;
if(ui->checkBox->isChecked()){
  patients = service.getPat1(...);
}else{
  patients = service.getPat2(...);
}
```

Work with selections: connect(ui->listWidget, &QListWidget::itemSelectionChang
```
auto selection = ui->listWidget->selectedItems();
if(selection.empty() return;
int index = ui->listWidget->currentRow();
```

Selection in QTableWidget:
```
QModelIndexList selection = ui->tableView->
->selectionModel()->selectedIndexes();
if(selection.empty()) return;
int row = selection.at(0).row();
std::string name = ui->tableView->model()->index(row
.data().toString().toStdString();
...
View* viewWindow = new View{service, name, ...};
viewWindow->show();
```

QString::number

Boolean evaluation:
```
bool status = std::stoi(statusString) == 1;
```

date validation:
```
if(date.length()!=10 || date[4]!='-' || date[7]!='-'){
    throw ...
}
for(int i=0; i<10; i++){
  if(i==4 && i!=7 && !std::isdigit(date[i])){
    throw ...
}}
```

Parse date: (with dot)
```
std::tuple<int,int,int> Service::parseDate(const std::string& dat
{
  int day=0, month=0, year=0;
  char dot1=0, dot2=0;
  std::stringstream ss(dateS);
  ss>>day>>dot1>>month>>dot2>>year;
  return std::make_tuple(year, month, day);
}
```

```
QFont font =
    item->font()
font.setBold(true
item->setFont(
    font);
```

Work with vector of pairs:
```
std::vector<Volunteer> Service::getTop(...)...{
  const auto& vol = getUnassignedVol();
  std::vector<std::pair<Volunteer, double>> result;
  std::vector<Volunteer> topVolunteers;
  for(auto v : vol) {
    double score = ...
    result.push_back({v, score});
  }
  std::sort(result.begin(), result.end(), [](
    const auto& a, const auto& b){
    return a.second > b.second;
  });
  for(const auto& pair : result){
    topVolunteers.push_back(pair.first);
  }
  if(topVolunteers.size()>3){
    topVolunteers.resize(3);
  }
  return topVolunteers;
}
```

std::vector<std::pair<Dep,int>> ... result.push_back({v,n}); std::sort(result.begin(), result.end(), [](const ... a.second > b.second);

Sort by date (chronologically) "YYYY-MM-DD (better)
```
std::sort(events.begin(), events.end(), [](
    const Event& e1, const Event& e2){
    return e1.getDate() < e2.getDate();
});
```

item->setBackground(Qt::green);

Remove (iterator usage!)
```
void Repo::remove(..description){
for(auto it = issues.begin(); it!=issues.end()
    it++){ if(...)
              throw ...
    issues.erase(it); save(); return;
}
throw ("not found"); }
```

```
void GUI::populateEventsList(){
  ui->...
  std::vector<Event> events;
  if(ui->checkBox->isChecked()){
    events = service.getEventsNearby(person);
  }else
    events = service.getSortedEvents(); ....
}...
```

Work with checkboxes: connect(ui->checkBox, &QCheckBox::toggled, this, &GUI::populateEventsList)

```
class Observer {
  public:
    virtual void update()=0;
    virtual ~Observer()=default;
};
```

```
class Subject {
  private:
    std::vector<Observer*> observers;
  public:
    Subject()=default;
    void registerObserver(Observer* obs);
    void unregisterObserver(Observer* obs);
    void notify();
};
```

```
class Service: public Subject
{ ....
} N
```
IF SERVICE DOES "NOTIFY", GUI DOES NOT DO UPDATE

```
void Service::add(...) {
  ....
  repo.add(...);
  ~~repo.save();~~
  notify();
}
```

```
void Repo::add(..){
  pat.push_back(p);
  save();
}
```

```
void Repo::load(){
  std::ifstream fin(filename);
  if(!fin.is_open()){
    throw std::runtime_error("...");
  }
  std::string line;
  while (std::getline(fin, line)){
    std::stringstream ss(line);
    std::string field1, ...;
    std::getline(ss, field1, 'sep');
    ....
    auto trim = ...
    trim(field1);
    ....
    std::vector<std::string> interests;
    std::stringstream lists(field3);
    std::string interest;
    while(std::getline(lists, interest, 'sep')){
      trim(interest);
      if(!interest.empty())
        interests.push_back(interest);
    }
  }
  volunteers.emplace_back(name, ....);
  fin.close();
}
```
convert string to a vector of strings

DeliverHowOOPExam → convert string to a pair of integers.

```
void Repo::save(){
  std::ofstream fout(filename);
  if(!fout.is_open()){
    ...
  }
  for(const auto& e: elements){
    fout << e.getName() << "|";
    fout << e.get....
    auto interests = e.getInterests();
    for(int i=0; i<interests.size(); i++){
      fout << interests[i];
      if(i!=interests.size()-1)
        fout << ",";
    }
    fout << "|";
    fout << e.get....
    fout << "\n";
  }
  fout.close();
}
```

```
class GUI : public QWidget, public Observer{
  public:
    explicit(Service& service, const Department& dep
    ~Gui() override;
    void update() override; → populateList()
  private:
    Ui::Gui *ui;
    Service& service; ← StarModel* model;
    Department dep; void updateFields();
    void connectSignalsAndSlots();
    void ...
```

```
observers.push_back(obs);
observers.erase(std::remove(observers.begin(), observers.end(),
  obs), observers.end());
```

```
Gui::Gui(...) : ..... {
  ui->setupUi(this);
  service.registerObserver(this);
  this->setWindowTitle(QString::fromStdString(dep.get...));
  ui->descriptionLabel->setText(QString:: ... );
  connectSignalsAndSlots();
  ~~Gui::~~update(); ← inside constructor!
}
```

```
Gui::~GUI(){
  service.unregisterObserver(this);
  delete ui;
}
```

```
void Gui::update(){
  populateList();
}
```
model->updateData(...)

```
int main(int argc, char* argv[]){
  QApplication app(argc, argv);
  Repo repo{"filename", ...};
  Service service{repo};
  std::vector<Gui*> windows;
  for(const auto& d: service.get...()){
    GUI* gui = new GUI{service, d};
    gui->show();
    windows.push_back(gui);
  }
  depWindow depWindow{service};
  depWindow.show();
  ~~int result = app.exec();~~
  return app.exec();
}
```

```
void Gui::add(){
  ....
  try{
    service.add(...);
    ~~update();~~
    ....->clear();
  }
  catch(...){
    ....
  }
```

count nr. of words in a string:
```
std::stringstream(desc)
std::string word;
int count=0;
while(ss>>word){
  count++;
}
return count;
```

QTABLE MODEL (gui->...)
```
class StarModel: public QAbstractTableModel{
  private: std::vector<Star> stars;
  public:
    StarModel(const std::vector<Star>& stars, QObject* parent=
    int rowCount(...) ... -> return stars.size(); nullptr);
    int columnCount(...) ... -> return 5;
    QVariant data(...)...
    QVariant headerData(...)...
    void updateData(const std::vector<Star>& newData);
};
```

```
StarModel::StarModel(...): QAbstractTableModel{parent}{
}
QVariant StarModel::data(...) ... {
  if(!index.isValid() || role!=Qt::DisplayRole) return QVariant{};
  const auto& s = stars[index.row()];
  if(index.column()==0) return QString::fromStdString(s.get...());
  ....
  if(index.column()==2) return s.getRA();
  ....
  return QVariant{};
}
```

```
void StarModel::updateData(...){
  beginResetModel();
  stars = newData;
  endResetModel();
}
```

QModelIndexList selected = ui->tableView->selectionModel()->selectedIndexes();